

Chapter 32 - IE Mon

IE Mon is the interactive monitor built into Intuition Engine. It runs in the same terminal as BASIC and gives you direct access to the bus, the active CPU's registers, breakpoints, watchpoints, a disassembler, an assembler-less memory editor, a reverse-execution timeline, save and restore of memory ranges, and a small handful of more specialised tools.

You enter the monitor from BASIC by typing `MON` at the prompt and leave it by typing `x`. Inside the monitor, each line is a command followed by space-separated arguments. Numeric arguments default to hexadecimal; prefix them with `&` for decimal or `%` for binary.

32.1 General conventions

Convention	Example	Meaning
Address	1000	Hexadecimal 0x1000
Address	&4096	Decimal
Address range	1000-10FF	Inclusive range
Register	r0, pc, a	Active CPU's register
Hex byte literal	7E	Hex byte for write/fill

The monitor's prompt is the active CPU's short name followed by a greater-than sign: (ie64)>, (6502)>, (z80)>, etc.

32.2 Execution control

Command	Argument(s)	Effect
g	[addr]	Resume execution from current PC or <code>addr</code>
s	[count]	Step: execute one instruction (or <code>count</code>)
u	addr	Run until: resume, stop one-shot at <code>addr</code>
x		Exit the monitor (return to BASIC)

s always re-prints the registers after the step. g returns to the monitor on the next breakpoint, watchpoint, or `Ctrl-C`.

32.3 Registers and disassembly

Command	Argument(s)	Effect
r	[reg] [value]	Show all registers, or set one register
d	[addr] [count]	Disassemble at <code>addr</code>
list	[addr] [count]	List source lines (when symbols are loaded)
cpu	[name]	Show the active CPU or switch to <code>name</code>

The argument to `cpu` is `ie64`, `ie32`, `6502`, `z80`, `m68k`, or `x86`.

32.4 Memory inspection and editing

Command	Argument(s)	Effect
m	addr [count]	Memory dump in hex + ASCII
w	addr byte [byte...]	Write bytes at addr
f	addr1-addr2 byte	Fill a range with one byte
h	addr1-addr2 pattern	Hunt: search a range for a byte pattern
c	addr1-addr2 dest	Compare a range against dest
t	addr1-addr2 dest	Transfer (block-copy) a range to dest

w enters a hex-edit minor mode if no bytes are supplied with the address: the monitor prompts for byte after byte until you hit Enter on an empty line.

There is no assemble command. To enter machine code, write the bytes with w and confirm them with d.

32.5 Breakpoints

Command	Argument(s)	Effect
b	addr	Set a code breakpoint at addr
bc	[addr id]	Clear a breakpoint by address or by ID
bl		List all breakpoints
bfirst		Break on the first hit only (set policy flag)

Breakpoints are sticky: setting one a second time at the same address increments a hit-counter rather than creating a duplicate.

32.6 Watchpoints (memory)

The monitor distinguishes between **byte**, **word**, **dword**, and **qword** watchpoints, and between **read**, **write**, and **read/write** flavours. The mnemonic encoding is:

bpm <size> <mode> where size = b/w/d/q and mode = r/w/(both)

Command	Effect
ww addr	Word watchpoint, read/write (legacy alias)
wr addr	Word watchpoint, read only
wrw addr	Word watchpoint, read only (verbose alias)
bpmb r addr	Byte read
bpmb w addr	Byte write
bpmb r/w addr	Byte read/write
bpmb r word addr	Word read

Command	Effect
bpmww addr	Word write
bpmw addr	Word read/write
bpmdr addr	Dword read
bpmdw addr	Dword write
bpmd addr	Dword read/write
bpmqr addr	Qword read
bpmqw addr	Qword write
bpmq addr	Qword read/write
wc [addr id]	Clear a watchpoint
wl	List all watchpoints

These are memory watchpoints: they do not write anything. The similar-looking ww/wr are the abbreviations sometimes confused with byte writers - they are not.

32.7 Reverse execution

The monitor records a per-instruction history of the active CPU's state. You can rewind, replay forward, and search the timeline.

Command	Argument(s)	Effect
bs	[count]	Back-step: undo one (or count) instructions
rs	[count]	Same as bs
rg		Reverse-continue: rewind until the previous breakpoint or watchpoint
rt	addr	Reverse-run-until: rewind until PC was addr
tl		Timeline: dump the recent history
history		Same as tl

Be careful: rg is **reverse-continue**, not "register inspect".

32.8 Tracing

Command	Argument(s)	Effect
trace	on off file	Enable/disable instruction trace
tracing	[size]	Enable a ring-buffer trace of recent instructions
show		Dump the ring trace

t trace is the named trace command. It is distinct from t (which is **transfer memory**, not trace).

32.9 CPU freeze and thaw

These commands stop and resume execution of a specific CPU without exiting the monitor.

Command	Argument(s)	Effect
freeze	cpuName *	Freeze one CPU, or * for all
thaw	cpuName *	Thaw one CPU, or * for all
fa		Freeze audio (mixer + all engines)
ta		Thaw audio

fa and ta are **audio-only** controls, not freeze-all aliases. To freeze every CPU at once, use freeze *.

32.10 Save and restore

Command	Argument(s)	Effect
save	addr1-addr2 file	Save a memory range to a sidecar file
load	addr file	Load a sidecar file at addr
ss	file	Save full machine state
sl	file	Load full machine state

save and load move memory ranges. ss and sl move the whole CPU + bus + device state. A saved-state file can be reloaded into a freshly reset system to resume from the same point.

32.11 Symbols, addresses, maps

Command	Argument(s)	Effect
sym	[name addr]	Look up a symbol or address
map		Show the active loaded symbol map
addr	addr	Resolve addr to a symbol if any

32.12 Bus, MMIO, page-level access

Command	Argument(s)	Effect
io	[addr]	Show MMIO region map or a specific page
pg	addr1-addr2 mode	Page guard: trap on access to a range
accesslog	on off	Enable per-page access logging
who	addr	Report the last CPU that accessed addr

32.13 Backtrace

Command	Argument(s)	Effect
bt		Stack backtrace from the active CPU

32.14 Quick reference card

The single-letter and short commands you will use most:

r	show registers	bt	backtrace
d [a]	disassemble	s [n]	step
m [a]	memory dump	g [a]	go
w a b...	write bytes	u a	run until
b a	breakpoint set	bs [n]	back-step
bc [a id]	breakpoint clear	rg	reverse-continue
bl	breakpoint list	rt a	reverse-run-until
ww/wr a	word watchpoint	tl	timeline
wc [a id]	watchpoint clear	x	exit
wl	watchpoint list	cpu n	switch CPU
freeze *	freeze all CPUs	thaw *	thaw all
fa / ta	freeze / thaw audio	save a-b f	save range
ss / sl f	save / load full state	load a f	load range at a

32.15 What comes next

Chapter 33 covers IE Script - a scripting language that drives the monitor from a file rather than from a terminal. IE Script is what you use to automate session setup, scripted breakpoint actions, data dumps after a fault, and similar batch debugging tasks.